Agentic RL on SWE-bench with slime and AgentCore

One fresh sandbox per episode, the agent loop inside it, and hidden tests as the reward; with EKS and ECS twins for comparison

Kaige Yang
Senior Applied Scientist, AWS

Data as of September 29, 2026

Agenda

01 The problem
What RL on SWE-bench asks of a sandbox

02 The system
slime + SGLang + a harness tier + one sandbox per episode

03 Three substrates
AgentCore and its limits; why EKS / ECS; compare and pick

04 Security
No public ports, no egress, no credentials; EKS hardening

05 How to run
Step-by-step recipe per substrate, gated creation, teardown

06 Status and next steps
What ran, what didn't, what's next

Appendix (from slide 28): knobs, references.

© 2026, Amazon Web Services, Inc. or its affiliates.

AgentCore runs the sandbox path cleanly; no issue solved yet

Setup	RL-train **Qwen3-4B** (slime async GRPO) on **SWE-bench Verified** issues. Each episode gets a fresh sandbox, and the reward is the row's hidden tests.
Works	First AgentCore run: **24 episodes, 0 infra errors**; the agent loop ran inside every sandbox; pre-open hid boot.
Not yet	**0** / **24 solved**; trainer CUDA OOM after 2 rollouts; 19 of 20 arm64 images still to build. EKS / ECS: code complete, **never deployed**.
Choice	**AgentCore** microVMs by default (420 / 500 rows); **EKS + gVisor** for the 80-row tail; **ECS** only as a trusted-code baseline.

Source: `code/demos/demo_cpu_swebench{,_eks,_ecs}/` READMEs; smoke log `logs/smoke_all_20260928_024714.log`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Every row has its own environment; every attempt needs a clean copy

Table 1 **The task, as an RL environment**

Observation	A real GitHub issue; the repo at `base_commit` in `/testbed`, conda env `testbed`
Actions	Tool loop: `bash`, `read_file`, `write_file` ($\leq$ 12 turns)
Reward	Hidden `test_patch` applied **after** the agent stops; 1 iff every F2P and P2P test passes
Dataset	SWE-bench Verified: **500** rows, 12 repos, 80 environments; demo subset 20 (16 train + 4 eval)

Source: `princeton-nlp/SWE-bench_Verified`; `demo_cpu_swebench/README.md`, `run.sh`; checked 2026-09-29.

- **One image per row:** no generic image serves the dataset.
- **GRPO multiplies it:** 8 siblings per row per step, each in a **pristine** copy that no sibling can touch.

© 2026, Amazon Web Services, Inc. or its affiliates.

Each episode needs its exact image, a fresh machine, and one way out

Table 2 **Per-episode requirements, met the same way on all three substrates**

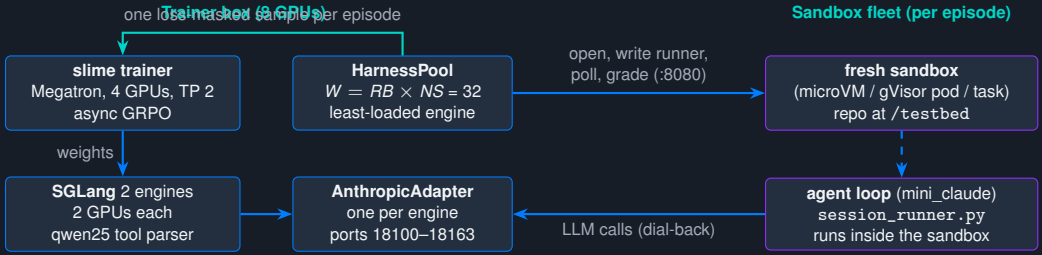
Need	Mechanism
The row's exact environment	One image per row, pinned by digest; the testbed env is never touched
Fresh state every episode	A brand-new session / pod / task, never reused
A route back to the policy	The in-sandbox loop dials the trainer's adapter over a private VPC path
Nothing else reachable	No egress except the adapter; no credentials (slide 21)
Hidden tests	`test_patch` stays on the trainer until grading
Lifecycle control	Pre-open, heartbeat, explicit stop, hard lifetime cap

Source: `demo_cpu_swebench/README.md` (design considerations), `swebench.py`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

slime on 4 GPUs, SGLang on 4, the agent loop inside the sandbox

Figure 1 **Architecture (one trainer box, 8 GPUs; sandboxes remote in the same VPC)**



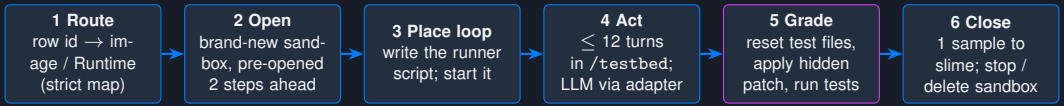
- **Harness in the sandbox:** tool calls stay local to the repo and only LLM calls cross the network, so throughput = min(inference, sandbox, harness). No in-trainer fallback (`HarnessPlacementError`).

Source: `demo_cpu_swebench/run.sh` (ACTOR_GPUS 4, ROLLOUT_GPUS 4, 2 GPUs per engine), `harness_pool.py`, `in_session_harness.py`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

One episode: route, open fresh, act inside, grade hidden, discard

Figure 2 **One episode, from slime's request to the training sample**



Grade	Reset files the `test_patch` touches to `base_commit`; `git apply` it; run the official per-repo `test_cmd`
Solved	Every F2P and P2P test PASSED / XFAIL, as in the official grader (no reinstall step)
Oracle check	Before training, per row: gold patch solves, empty patch doesn't (`usable=1`)
Pre-open	The look-ahead boots the next 2 steps' sandboxes, so boot leaves the critical path
Heartbeat	AgentCore: pings beat the 15 min idle reclaim. EKS / ECS: liveness; dead or near-expiry pre-opens are replaced

Source: `demo_cpu_swebench` README, `swebench.py`, `terminalbench_pool.py`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Fully-async GRPO: 4 rows $\times$ 8 samples per step, no KL

Table 3 **slime settings (`run.sh` defaults)**

Policy	Qwen3-4B, TP 2 (Megatron)
Driver	slime `fully_async_rollout`
Estimator	GRPO, $RB = 4$, $NS = 8$ (32 per step)
Clip / KL	ϵ 0.2 / 0.28; KL coef 0
LR	1×10^{-6}, constant
Per turn	$\leq$ 1024 tokens; tool output clipped to 2000 chars
Per episode	$\leq$ 12 turns; bash timeout 180 s
Tokens per GPU	24,576 ($\geq$ one full trajectory)
Eval	every 10 steps, 4 held-out rows $\times$ 2

- **Tool parser is mandatory:** without `-sglang-tool-call-parser qwen25` the loop never sees a tool call and reward is structurally 0.
- **Admission:** $CONC = \lceil 2W / N_{eng} \rceil$, so a free worker never waits for an engine.

Source: `demo_cpu_swebench/run.sh`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Demo 1, AgentCore: one arm64 microVM per episode, no fleet to run

Use case covered	RL on real GitHub issues with the strongest per-episode wall: `demo_cpu_swebench`, 20 rows, one VPC microVM Runtime per row.
Why this demo	SWE-bench is the extreme of "one image per row". It shows the managed fleet hosts it as is: no capacity provider, nothing billed idle.
Design considerations	**arm64 only:** Epoch AI rebuilds cover **420** / **500** rows. **2 GB cap:** 2048 MiB *compressed*, probed on AWS; every arm64 image fits (max 1.73 GB). **Wrapper:** standalone Python + HTTP server in `/opt/acs`; testbed untouched. **Network:** VPC mode, egress none, SG→SG dial-back. **Lifecycle:** new `runtimeSessionId` per episode, `StopRuntimeSession` at the end.

The 80 rows without an arm64 image need a rebuild, the retired m5 Instance lane (≈ 90 s boot), or EKS (slide 12).

Source: `demo_cpu_swebench/README.md`; `survey_image_sizes.py`, `probe_image_cap.py` → `swebench/capprobe*.csv`; us-west-2, 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

A cold microVM opens in 22–30 s near the cap, $\approx 3\times$ faster than m5

Table 4 **Session open, measured on AWS**

microVM, warm	0.4–1.0 s (about 10 per new Runtime found pre-warmed)
microVM, cold	23.5 s at 1.5 GiB; **28–30 s** at 1.95–2.04 GiB
m5 Instance lane	p50 **93 s** (pull + boot p50 91 s)
Boot on critical path	**p50 0** with pre-open (max 143 s, first batch)

microVM numbers: synthetic probe images (54 opens). m5: the SWE-bench smoke run (24 episodes).

- The arm64 SWE-bench images are 0.7–1 GB, under the probed sizes; their open time is **not measured yet**.
- Pre-open hid even the 90 s m5 boot, so a 30 s boot hides easily.

Source: `probe_image_cap.py` (2026-09-28); `demo_cpu_swebench/logs/smoke_all_20260928_024714.log`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

AgentCore limits: arm64 + 2 GB leave 80 rows out; one Runtime per row

Table 5 **Limits of the microVM path, each verified on AWS or in the docs**

Limit	Effect on SWE-bench	Mitigation
arm64 only [D1]	**80** / **500** rows have no arm64 build: 27 matplotlib, 25 sklearn, all 22 xarray, 4 astropy, 2 django	Rebuild; EKS / ECS
2 GB image cap [P1]	2048 MiB *compressed*, not adjustable, checked at `CreateAgentRuntime`	Select $\leq$ 1.9 GB; else `too_large`
One Runtime per row [Q1]	1000 per account; build, push, register, oracle check all scale with rows	One image per env: 500 $\rightarrow$ 80
Cold open per episode	22–30 s near the cap; a new session every episode, so no warm reuse	Pre-open 2 steps ahead
Idle reclaim, quotas [Q1][Q2]	Idle $\approx$ 15 min reclaims a session; 5000 active, 25 new / s, 8 h max	Heartbeat (on); far above 64
VPC dial-back	The in-session loop must reach the trainer's adapter	SG$\rightarrow$SG rule 18100–18163

Source: AgentCore docs [D1][Q1], service quotas [Q2], cap probe [P1], `survey_image_sizes.py`; checked 2026-09-28/29.

© 2026, Amazon Web Services, Inc. or its affiliates.

EKS and ECS twins cover the 80 rows and price the managed sandbox

1. Coverage

Stock amd64 images with no size cap run **all 500 / 500** Verified rows, including xarray and the large matplotlib images.

2. Comparison

Same rows, harness, grader, recipe and metrics; **only the sandbox differs**. Open latency, cost, isolation and ops burden side by side, for teams that already run EKS or ECS.

In plain English: each twin swaps one class (`EksPodSandbox` / `EcsTaskSandbox`) and shares the ECR images and the :8080 server; everything else is identical.

Source: `demo_cpu_swebench_eks/README.md`, `demo_cpu_swebench_ecs/README.md` (Why this demo); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Demo 2, EKS: one fresh gVisor pod per episode on shared m5 nodes

Use case covered	The same workload on self-managed Kubernetes: `demo_cpu_swebench_eks`, same 20 rows, stock amd64 images.
Why this demo	Answers "what would this cost on EKS?": all 500 rows run, ≈4 episodes per m5.2xlarge, and gVisor keeps it an honest sandbox.
Design considerations	**Isolation:** `runtimeClassName: gvisor` (`runsc`, systrap, no KVM): a user-space kernel per pod. **Transport:** HTTP straight to `podIP:8080` (VPC CNI); no `kubectl`. **Images:** per-node cache + a pre-pull DaemonSet; 300 GiB gp3 per node. **Sizing:** pod 1.5 vCPU / 6 GiB; 4 nodes ≈ 16 pods; pre-open capped to free slots. **Network:** NetworkPolicy: in :8080 from the trainer, out to the adapter only.

Source: `demo_cpu_swebench_eks/README.md`, `eks_pod_sandbox.py`, `deploy/eks/`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

EKS pros and cons: every image and dense packing, but you run it all

Pros

- ✓ **Every row:** any amd64 image, any size; **500** / **500** rows incl. the 80 AgentCore can't take
- ✓ **Stronger wall than ECS:** gVisor puts a user-space kernel under each pod
- ✓ **Dense:** ≈ 4 pods per m5.2xlarge vs one microVM per episode
- ✓ **Fast cached opens** (2–6 s[est]): a pre-pull DaemonSet loads every image per node
- ✓ **Tunable security:** NetworkPolicy, per-pod token, IMDS off, private API, pinned gVisor
- ✓ **Fits EKS shops:** no per-row Runtime, no 1000-Runtime cap

Cons

- × **You operate it all:** control plane, nodes, gVisor installer, policy, upgrades
- × **Idle cost:** control plane + nodes bill with no episode running
- × **gVisor tax:** syscall-heavy suites run slower, some may break (oracle check on gVisor)
- × **Shared node:** CPU, memory bandwidth, page cache, disk; weaker than a microVM
- × **Manual capacity:** no autoscaler; 4 nodes = 16 slots $<$ 32 running, so pre-open is off
- × **Unproven:** tested offline only; the cluster was never created, all numbers are estimates

In plain English: use EKS for the rows AgentCore can't take, or if you already run EKS; not as the default.

Source: `demo_cpu_swebench_eks/README.md` (limitations, known first-run risks); [est] = not measured; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Demo 3, ECS: one fresh task with its own ENI, the simplest baseline

Use case covered	The same workload on ECS on EC2, no Kubernetes: `demo_cpu_swebench_ecs`.
Why this demo	The baseline most teams already run (`RunTask` / `StopTask` + an ASG); shows what a managed sandbox buys over it.
Design considerations	**Isolation:** plain runc, **node kernel shared**; all caps dropped, no task role, IMDS blocked. **Network:** `awsvpc`, own ENI + SG per task; in :8080 from the trainer, out to the adapter only. **Scaling:** capacity provider, managed scaling, ≈ 4 tasks per node, up to 16 nodes. **Polling:** one thread, batched `DescribeTasks`.

Source: `demo_cpu_swebench_ecs/README.md`, `ecs_task_sandbox.py`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

ECS pros and cons: the simplest fleet, behind the thinnest wall

Pros

- ✓ **Simplest self-managed path:** `RunTask` / `StopTask` + an ASG; ECS itself is free
- ✓ **Every row:** any amd64 image, any size; **500** / **500** rows
- ✓ **Own ENI + SG per task:** network rules per episode, not per node
- ✓ **Native runc speed:** no gVisor syscall tax; cached opens 5–15 s[est]
- ✓ **Autoscales:** capacity provider with managed scaling, 0–16 nodes
- ✓ **Pure boto3:** no eksctl, no Kubernetes, one idempotent script

Cons

- × **Node kernel shared:** a kernel exploit reaches the node; **trusted models only**
- × **Hardening is partial:** caps dropped, no task role, IMDS blocked, but no syscall sandbox
- × **No per-task disk quota;** `RunTask` throttling at high open rates
- × **ENI attach** adds seconds per open; ENI trunking needed for density
- × **You operate:** ASG, capacity provider, launch template, one task def per image
- × **Unproven:** never deployed; the trainer role lacks `ecs:*`

In plain English: use ECS as a cost / latency baseline or for trusted-model runs; never for untrusted RL code at scale.

Source: `demo_cpu_swebench_ecs/README.md` (limitations); [est] = not measured; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

AgentCore isolates best with no ops; EKS / ECS run any image, denser

Table 6 The three substrates on the same workload (est = estimate, not measured)

	AgentCore (arm64 microVM)	EKS + gVisor pod	ECS task (runc)
Isolation	**own microVM** per episode	user-space kernel; node shared	container; **node kernel shared**
Images	arm64, $\leq$ 2 GB; 420 / 500 rows	any amd64 image; 500 / 500	any amd64 image; 500 / 500
Open	cold 22–30 s (measured)	cached 2–6 s^{est}; pull 30–90 s^{est}	cached 5–15 s^{est}; pull 30–90 s^{est}
Close / leak cap	`StopRuntimeSession`; idle reclaim	`DELETE` pod; deadline 2 h	`StopTask`; `timeout` 2 h
Density	1 microVM per episode	$\approx$ 4 pods per m5.2xlarge	$\approx$ 4 tasks per m5.2xlarge
Pre-open room	5000 sessions	16 slots at 4 nodes: **off** until 16 nodes	64 slots at 16 nodes: 32 pre-open
You operate	Runtimes; nothing idle	cluster, nodes, gVisor, policy	ASG, capacity provider, task defs
Best for	the 420 eligible rows	the 80-row tail; EKS shops	trusted-model baselines
Watch out	one Runtime per row	gVisor syscall tax; ops	shared kernel
Status	m5 lane ran; arm64 pending	not deployed	not deployed

Source: slides 9–16; EKS / ECS open times are design estimates, not measurements; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Launch time: AgentCore registers per row; EKS / ECS pay per node

Table 7 Time to launch an image (once) and a sandbox (every episode); est = estimate, not measured

	AgentCore (arm64 microVM)	EKS + gVisor pod	ECS task (runc)
Per image, once			
Build + push	arm64 wrap under qemu: 56 s (1 row); push 1.9–2 GB: 128–134 s	amd64 wrap: p50 **110 s** build, 8 s push (19 rows, m5 lane)	same images as EKS: no rebuild
Register	`CreateAgentRuntime` → READY **21 s** per row	none; pre-pull to every node, 1–3 min[est]	task definition, ≈ 1 s[est]; optional pre-pull
Fleet	none	cluster + nodes 15–20 min	ASG + nodes 3–5 min[est]
Per sandbox, every episode			
Best case	warm 0.4–1.0 s (first ≈ 10 sessions of a Runtime)	image cached: 2–6 s[est]	image cached: 5–15 s[est] (ENI attach)
Typical	cold **22–30 s** near the cap (pull is inside boot)	not cached: 30–90 s[est] pull	not cached: 30–90 s[est] pull
Out of room	none below 5000 sessions, 25 new / s	Pending until a slot frees (no autoscaler)	PROVISIONING while a node boots, +2–4 min[est]
On critical path	**p50 0 s** with pre-open (m5 lane)	0 if pre-open fits the slots	0 if pre-open fits the slots

Source: `swebench/images.csv`, `m5_legacy/images.csv`, `capprobe*.csv` (us-west-2, 2026-09-28); EKS / ECS READMEs; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Termination: every episode is killed explicitly, and every leak is capped

Table 8 How a sandbox ends on each substrate

	AgentCore (arm64 microVM)	EKS + gVisor pod	ECS task (runc)
Episode ends	`StopRuntimeSession`: the microVM is destroyed	`DELETE` pod, grace 0: SIGKILL, gVisor sandbox torn down	`StopTask`: SIGTERM, SIGKILL after 2 s; ENI released
Never reused	new `runtimeSessionId` per episode	new pod name per episode	new task per episode
Bad pre-open	heartbeat prevents reclaim; a reclaimed one cold-boots again on first use	dead or near-expiry pods replaced before claim	dead or near-expiry tasks replaced before claim
Job ends	pool closes every open and pre-opened session	same: every pod deleted	same: every task stopped
Trainer crash (leak)	idle reclaim after ≈ 15 min (no heartbeat); 8 h max	`activeDeadlineSeconds` 7200 s; kubelet kills it	entrypoint `timeout -s KILL` 7200 s
Manual sweep	`6_cleanup.sh` deletes the Runtimes	delete label `app=acs-swe-sandbox`; `EKS_DELETE=1`	`down` stops `startedBy=acs-swe`; `ECS_DELETE=1`
Billing after end	none	nodes + control plane until deleted	nodes until scaled in or deleted

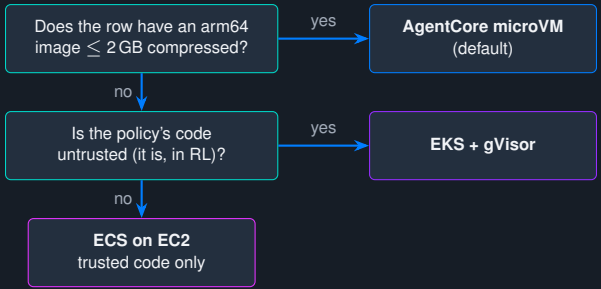
In plain English: on AgentCore a leak costs at most 15 idle minutes; on EKS / ECS it holds a slot for up to 2 h, and the nodes bill until you delete them.

Source: `agentcore_runtime_sandbox.py`, `eks_pod_sandbox.py`, `ecs_task_sandbox.py`, `ecs_deploy.py` (`stopTimeout` 2), `terminalbench_pool.py`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Start on AgentCore; move a row to EKS only when its image rules it out

Figure 3 **Decision guide for one SWE-bench row**



- **AgentCore:** strongest wall, nothing to operate, no idle cost.
- **EKS + gVisor:** xarray, big matplotlib, > 1000 environments, or an existing EKS fleet.
- **ECS:** a cost / latency baseline for trusted models; the kernel is shared.

Source: slide 17; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

No public port: in on :8080 from the trainer, out only to the adapter

Figure 4 Network paths (all private, one VPC)

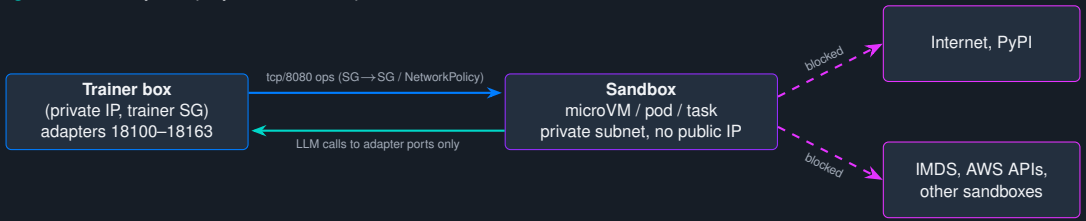


Table 9 Controls common to all three substrates

Inbound	Only tcp/8080 from the trainer's SGs; no load balancer, no public IP
Outbound	Only the adapter ports (18100–18163, 18021): AgentCore `RUNTIME_EGRESS=none`; EKS NetworkPolicy; ECS task SG
Credentials	None in the sandbox: no AWS role or IMDS, no Kubernetes service-account token
Grader	The hidden `test_patch` never enters the image or the dataset row

Source: `agentcore_deploy.py`, EKS NetworkPolicy, ECS README; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

EKS hardening: eight controls between agent code and node, cluster, AWS

Table 10 **EKS threat → control (implemented and tested offline; not yet run on a cluster)**

Threat	Control
Kernel exploit from agent code	**gVisor** (`runsc`): every syscall hits a user-space kernel; only on nodes labelled `acs/gvisor=ready`
Privilege escalation in the pod	`allowPrivilegeEscalation: false`, `capabilities.drop: [ALL]`, `seccompProfile: RuntimeDefault`
Stealing the node's AWS role	IMDSv2 only, hop limit 1 (`disablePodIMDS`); egress policy also blocks 169.254.169.254
Talking to the Kubernetes API	No service-account token, no service links; API endpoint **private-only** after create
Another host calling :8080	NetworkPolicy + SG allow only the trainer, plus a **per-pod random token** on every request
Lateral movement, exfiltration	Egress only to the adapter ports: no DNS, no internet, no other pod
Tampered gVisor download	Release **pinned** and checked against its sha512 before install
Leaked or runaway pods	`activeDeadlineSeconds` 7200 s; memory limit 8 GiB; 10 GiB ephemeral

Residual risk: pods share a node's CPU, memory bandwidth and page cache; the one-time gVisor installer DaemonSet is privileged.

Source: `demo_cpu_swebench_eks/` (`eks_pod_sandbox.py`, `deploy/eks/`); gVisor docs [G1]; VPC CNI network policy [E1]; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

All three demos run the same six numbered scripts

Table 11 **The shared recipe (`cd code/demos/<demo>`; container steps `docker exec` themselves)**

Script	What it does
`1_prepare_data.sh`	`fetch_swebench.py` picks the 20 rows → `swebench/manifest.json` (holds the hidden patches) → `data/*.jsonl`
`2_build_image.sh`	`deploy_swebench.py`: wrap + push **one image per row** to ECR (resumable by recipe tag)
`3_push_ecr.sh`	No-op: prints `swebench/images.csv` (step 2 already pushed)
`4_deploy_runtime.sh`	Create the sandboxes (Runtimes / cluster / ECS), **oracle check** every row, keep usable rows only
`5_run.sh`	slime: 10 train steps + 1 eval → `logs/<tag>.{log,csv,eval.csv,png}`
`6_cleanup.sh`	Tear down what step 4 created (ECR images are kept)

- **Prerequisites:** the `agentcore_p4` trainer container (repo at `/work/code`), 8 GPUs, AWS credentials; `eksctl` for EKS; ECS / EC2 / ASG / IAM rights for ECS.
- **Always validate on 2 rows first** with `SWE_LIMIT=2` on steps 2 and 4 (real AWS cost).

Source: `deploy/demo.env` + `deploy/steps_common.sh` in each demo; READMEs (Step by step); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

AgentCore recipe: no fleet; step 4 registers one Runtime per row

```
cd code/demos/demo_cpu_swebench
bash 1_prepare_data.sh
# validate on 2 rows first (billed)
SWE_LIMIT=2 bash 2_build_image.sh
SWE_LIMIT=2 bash 4_deploy_runtime.sh
bash 5_run.sh
bash 6_cleanup.sh # delete Runtimes

# no AWS at all: build + oracle under qemu
SWE_NO_PUSH=1 SWE_TASKS=<id> \
  python3 deploy/deploy_swebench.py
SWE_TASKS=<id> \
  python3 swebench_oracle_check.py --local
```

Table 12 What it creates, and how to check it worked

Step 2	ECR `acs-swe-arm64-*`: arm64 image + server (buildx + qemu)
Step 4	VPC Runtimes `acs_swearm_*`, egress none; images > 2048 MiB marked `too_large`
Cost	microVM time only; nothing idle
Verify	`oracle.csv`: `usable=1`; log: `fresh_session=1`, `infra_error=none`
Clean	Step 6 deletes every Runtime in the map

Source: `demo_cpu_swebench/README.md`, `deploy/demo.env`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

EKS / ECS recipe: nothing is created until `*_CREATE=1`

Table 13 **EKS (`demo_cpu_swebench_eks`)**

```
bash 1_prepare_data.sh
SWE_LIMIT=2 bash 2_build_image.sh
python3 deploy/eks_deploy.py render # read-only
# billed: control plane + nodes, ~15-20 min
EKS_CREATE=1 bash 4_deploy_runtime.sh
python3 deploy/eks_deploy.py smoke
bash 5_run.sh
EKS_DELETE=1 bash 6_cleanup.sh
```

Table 14 **ECS (`demo_cpu_swebench_ecs`)**

```
bash 1_prepare_data.sh
SWE_LIMIT=2 bash 2_build_image.sh # reuses EKS images
python3 deploy/ecs_deploy.py render # read-only
# billed: EC2 nodes (ECS itself is free)
ECS_CREATE=1 bash 4_deploy_runtime.sh
python3 deploy/ecs_deploy.py smoke
bash 5_run.sh
ECS_DELETE=1 bash 6_cleanup.sh
```

	EKS	ECS
Creates	`eksctl` cluster (4 × m5.2xlarge), gVisor DaemonSet, RuntimeClass, NetworkPolicy, pre-pull	role, SGs, launch template, ASG, capacity provider, cluster, one task def per image
Smoke	pod shows gVisor, prints the testbed Python; oracle check *on gVisor*	`CapEff` zero, IMDS and internet unreachable, testbed Python
Clean	revoke SG rules, delete cluster; ECR kept	stop tasks, delete everything above; ECR kept

Source: `demo_cpu_swebench_{eks,ecs}/README.md`, `deploy/{eks,ecs}_deploy.py`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

The first run proved the sandbox path, not the learning

Table 15 **Where each demo stands**

AgentCore, m5 lane (retired)	Smoke: **24 episodes, 0 infra errors**, 12-turn loops ran in-session. **0** / **24 solved**; 28 of 48 turn stops hit the turn budget. Trainer **CUDA OOM** (log-softmax, 6.8 GiB) after 2 rollouts
AgentCore, arm64 microVM	**1** / **20** images built, **locally only**; oracle passes under qemu. Not pushed, no Runtime yet
EKS + gVisor	Code complete and hardened (slide 22); pre-open capped to node slots, dead / near-expiry pre-opens replaced; tested offline; cluster **never created**
ECS on EC2	Code complete, same pre-open healing; tested against fake AWS clients; **never deployed**; trainer role lacks `ecs:*`

In plain English: the machines were ready on time every time; the 4B model just hasn't fixed a bug yet.

Source: `demo_cpu_swebench/logs/smoke_all_20260928_024714.log`, `swebench/images.csv`, `oracle_local.csv`; EKS / ECS READMEs; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Next: arm64 Runtimes and the OOM, then a denser reward

Table 16 **Next steps, in order**

Step	Why
1. Build, push, register the other 19 arm64 images; oracle-check all 20 on AWS	Only usable rows train
2. Fix the trainer OOM (`expandable_segments`, lower `MAX_TOKENS_PER_GPU`)	Get past step 2
3. Smoke run on the microVM lane	Real open times
4. Easier rows, a curriculum, or more turns	0 solves = no gradient
5. Tear down the m5 pool (`m5_legacy`)	Stop idle cost
6. EKS / ECS, on request: create on 2 rows, oracle check, smoke	Measure slide 17

Source: Project status, 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Appendix

Knobs · references

Knobs worth knowing

Table 17 **Environment variables (defaults in each demo's `run.sh` / README)**

`RB / NS / NR`	rows per step / GRPO group size / rollouts (4 / 8 / 11 = 10 train + 1 eval)
`EPISODE_MAX_TURNS / EPISODE_MAX_TOKENS`	12 turns / 1024 tokens per turn
`TB_PREOPEN_BATCHES / HEARTBEAT`	steps pre-opened ahead (2) / keeper ping (1)
`WARM_IDLE_TIMEOUT`	AgentCore 900 s; EKS / ECS 240 s (liveness ping every $\approx$ 2 min)
`SANDBOX_SLOTS`	EKS / ECS fleet size (nodes $\times$ 4); pre-open $\leq$ slots $-$ workers; 0 = no cap
`TB_EPISODE_BUDGET_SEC`	pre-open used only if an episode still fits the 7200 s pod / task lifetime
`MAX_TOKENS_PER_GPU`	24576 (the smoke run OOMed here)
`SWE_LIMIT / SWE_TASKS / SWE_MAX_IMAGE_MB`	limit / pick rows / size filter (1900)
`RUNTIME_EGRESS`	AgentCore: `none` (default) or `nat`
`EKS_RUNTIME_CLASS / EKS_NODES`	`gvisor` (empty = runc, A/B only) / 4 nodes

Source: Demo READMEs and `run.sh`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

References

Ref	Source	Supports
[D1]	`https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html`	Session = dedicated microVM; linux/arm64 images
[Q1]	`https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/bedrock-agentcore-limits.html`	1000 Runtimes; 15-min idle timeout
[Q2]	`aws service-quotas -service-code bedrock-agentcore` (us-west-2, 2026-09-28)	2048 MB image (not adjustable); 5000 active, 25 new sessions / s
[P1]	Project probe `probe_image_cap.py` (us-west-2, 2026-09-28)	2048 MiB compressed cap; open times
[G1]	`https://gvisor.dev/docs/`	User-space kernel per sandbox
[E1]	`https://docs.aws.amazon.com/eks/latest/userguide/cni-network-policy.html`	VPC CNI network policies
[S1]	`princeton-nlp/SWE-bench_Verified`; GHCR `epoch-research/swe-bench.eval.arm64.*`	500 rows; arm64 rebuilds
[R1]	slime: `https://github.com/THUDM/slime`	Async GRPO trainer, SGLang rollout

Source: Links checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Thank you

Kaige Yang · Senior Applied Scientist, AWS